

파이썬 프로그래밍 3주차 요약본

11. tkinter

11.1 tkinter

- 파이썬 표준 라이브러리
 - 윈도우 위젯 생성 라이브러리

```
import tkinter
```

- 윈도우 위젯(widget, 작은 프로그램, 객체) 생성
 - tkinter 라이브러리 클래스 중 윈도우 객체를 생성하는 Tk 클래스 사용
 - Tk 클래스가 가장 상위층(Toplevel) 위젯을 생성

```
class tkinter.Tk(  
    screenName=None,  
    baseName=None,  
    className='Tk',  
    useTk=1  
)
```

11.2 tkinter 라이브러리의 함수들

- 위젯의 버튼, 해당 위젯의 기능(이벤트), 객체 등을 정의 및 실행

```
위젯 클래스 객체 = 위젯(  
    부모가 되는 인스턴스,  
    option1 = xxxx,  
    option2 = xxx,  
    .....  
)  
위젯클래스.pack(option)
```

```
위젯클래스.grid(option)
위젯클래스.place(option)
tkinter.Button(command = 함수명)
```

12. 파일 처리

12.1 파일 열기

- 파일 열기(생성)

```
f = open("F:/rokey/ch12/file1.txt",'w')
```

- 파일 열기 모드
- r : 읽기 모드, w : 쓰기 모드, a : 추가모드
- 인코딩
 - 데이터를 컴퓨터가 이해할 수 있는 바이너리(이진법 0, 1) 형식으로 변환하는 것(예 아스키 ASCII, 유니코드 등)
- 디코딩
 - 바이너리 형식에서 사람이 이해할 수 있는 문자로 변환하는 것

12.2 파일 닫기

- close() 메서드

```
f = open("F:/rokey/ch12/file1.txt",'w')
f.close()
```

12.3 파일 쓰기

- write() 메서드

```
f = open("F:/rokey/ch12/file1.txt",'w')
for i in range(1, 11):
    data = "%d번째 줄입니다.\t"%i
```

```
f.write(data)
f.close()
```

12.4 파일 읽기

- `readline()` 메서드 파일을 읽어서 변수에 저장 후 반환

```
path = "./ch12/file1.txt"
f = open(path, 'r')
line = f.readline()
print(line)
f.close()
```

- `readlines()` 메서드 파일을 읽어서 변수에 리스트로 저장 후 반환

```
path = "./ch12/file1.txt"
f = open(path, 'r')
lines = f.readlines()
for i in lines:
    print(i, end='')
f.close()
```

- `read()` 메서드 파일을 읽어서 변수에 문자열로 저장 후 반환

```
path = "./ch12/file1.txt"
f = open(path, 'r')
data = f.read()
print(data)
f.close()
```

12.5 파일 내용 추가

- 기존 데이터를 유지하면서 새로운 값을 추가

```
path = "./ch12/file1.txt"
f = open(path, 'a')
for i in range(11, 20):
    data = '%d번째 줄입니다.\n'%i
```

```
f.write(data)
f.close()
```

12.6 자동으로 파일 닫기

- 자동 파일 닫기
- 파일을 열고 닫는 것을 자동으로 처리

```
path = "./ch12/file1.txt"
mode = 'w'
with open(path, mode) as f:
    f.write('No pain, no gain')
```

+파일개요

- **파일(File)** : 컴퓨터에서 데이터를 저장하는 기본 단위
- 컴퓨터 메모리에서 시스템 영역, 데이터 영역이 있는 것을 기억해보자, 파일은 데이터 영역
- 파일 이름과 확장자로 식별
- 파일을 호출, 저장하는 경로가 필요
- 현재 작업 디렉토리를 확인하는 파이썬 코드

```
import os
print(os.getcwd())
```

13. 예외 처리, 문자열, 람다 함수, map 함수

13.1 예외 처리

- 문법적으로 문제가 없으나 실행 과정에서 생기는 예외에 대한 실행을 정의
- 예외의 경우 (사용자의 돌발 행동, 기기의 물리적 이슈에 의한 경우 등등)

13.2 try....except

- 리스트 메모리 활용

```
while True:
    try:
        x = int(input('Please enter a number:'))
        break
    except ValueError:
        print("That's wrong!")
```

13.3 예외 일으키기

- raise 문
- 프로그래머가 지정한 예외가 발생하도록 강제

```
try:
    raise NameError('Hi')
except NameError:
    print('An exception flew by!')
```

13.4 문자열

- 수정이 불가능
- 인덱스와 길이를 가진다(인덱싱과 슬라이싱 가능)

```
l = '두산로키부트캠프'

len(l) #문자열의 길이
l[:3] #문자열의 인덱스 활용
```

- 문자열 + 문자열 이 가능
- f-string

```
i = 30

print(f'10+15 = {i}')
```

13.5 인덱싱과 슬라이싱

- 리스트, 튜플, 문자열 등 인덱스를 가지는 자료구조, 자료형은 자동으로 0부터 시작하는 인덱스를 가짐
- 맨 오른쪽부터 시작하는 인덱스는 -1부터 시작
- 슬라이싱 []와 콜론(:)을 사용하여 해당 자료형, 자료구조의 부분 집합을 추출

```
i = [1, 2, 3, 4]
```

```
i[1:3] #해당 값은 [2, 3]
```

13.6 람다 함수

- 파이썬의 요약 문법
- 간단한 함수를 def 로 정의하지 않고 하는 문법
- 필수적으로 반드시 써야 하는 것은 아니지만 해당 문법이 사용된 코드를 봤을 때 이해할 수 있어야 함
- 아래 예제의 정의된 함수와 lamda 식은 같은 함수

```
def fun(x):  
    return x+x
```

```
lamda x : x+x
```

13.7 map() 함수

- 반복적인 자료 구조(리스트, 튜플 등)의 요소 하나하나에 어떤 함수를 적용해서 리스트로 반환

```
def squaer(x):  
    return x ** 2
```

```
exampe = [1,3,5,7]  
result = map(squaer, exampe)
```

14. 정규표현식

14.1 메타문자

- 파이썬 내에서 어떤 자료 구조나 특정 표식을 위해 사용되는 기호, 특수 문자들
- 리스트를 표현할 때 쓰는 [], 튜플을 표현하는 () 등

14.2 정규표현식이란

- 정한 패턴을 이용하여 문자열을 검색, 일치 여부 확인, 추출 및 변환할 수 있도록 하는 방법
- re 모듈을 사용하여 정규 표현식을 컴파일 후 모듈 내의 함수를 활용

```
import re
p = re.compile("^python\s\w+")

data = """python one
life is too short
python two
you need python
python three"""

print(p.findall(data))
```

15. 고급 함수

15.1 이터레이터

- 이터레이터: next 함수 호출 시 계속 그 다음 값을 리턴하는 객체
- 리스트는 이터레이터 아님(이터레이터로 변환은 가능)

```
# iterator.py
class MyIterator:

    def __init__(self, data):
```

```

self.data = data
self.position = 0

def __iter__(self):
    return self

def __next__(self):
    if self.position >= len(self.data):
        raise Stopiteration
    result = self.data[self.position]
    self.position += 1
    return result

if __name__ == "__main__":
    i = MyIterator([1,2,3])
    for item in i:
        print(item)

```

15.2 제너레이터

- 이터레이터 생성 함수
- 차례대로 결과를 반환하고자 return 대신 yield를 사용

```

def mygen():
    for i in range(1, 1000):
        result = i*i
        yield result

gen = mygen()
print(next(gen))
print(next(gen))
print(next(gen))

```


